LESSON PLAN · PYTHON (GR 6–8) INTERMEDIATE: PYTHON GAME CODERS · LEVEL 2

Lesson 7: Records and Classes: Virtual Pet

Module 2 | 60 minutes | lists of dicts, classes | Grades 7–11 · ages 12–16

Objective: Store records in a list of dictionaries, write your first class with attributes and methods, and build a virtual pet game.

Success criteria

- I can store and loop over records in a list of dictionaries.
- I can find a total and a top value in a list of records.
- I can write a class with __init__, attributes and methods.
- I can create objects and call their methods.

Key vocabulary

- **record**: One item of data with several parts, like one player's dictionary.
- **list of dictionaries**: A table of records: one dictionary per row.
- **class**: A blueprint that describes data and actions together.
- **object**: One real thing built from a class, like Pet("Moti").
- **attribute**: A value an object remembers, like self.hunger.
- **method**: A function inside a class, called like pet.feed().

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Project files: team_sheet.py

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Pet Factory**	Play Pet Factory (unplugged): learners become pet objects and update only their own numbers when called.
5 min	**Hook and video**	Show the hook. Chat poll: "Name a pet for our class object!" Play the video with captions; pause at "A class is a blueprint".
10 min	**Live demo and build-along**	Run the live demo, then everyone types the cricket team sheet and the Player class. Thumbs check: "Did Meera reach 10 runs?"
8 min	**Challenge, predict and order**	Team manager challenge, the batting object prediction and the class line order. Quick competition: first correct order in the chat wins.

15 min	**Virtual pet in breakout pairs**	Pair programming in breakout rooms: one partner writes the methods, the other writes the game loop; swap and test with feed, play, dance and quit.
6 min	**Debug race**	Race to fix “the forgetful pet”. The winner explains what self does.
7 min	**Pet show-and-tell**	Three learners introduce their pet live; the class types commands in the chat and the owner runs them. Vote for the happiest pet.
4 min	**Wrap-up and exit ticket**	Fist-to-five on the success criteria. Exit ticket: self.name question.

Unplugged warm-up

Pet Factory (5 min game): the teacher draws a “Pet blueprint” card on the board with three attributes (hunger, happiness, energy) and three buttons (feed, play, sleep). Three learners each “become” a pet object with their own name and start every number at 5 on paper. The class calls “Moti.feed()!” or “Bijli.play()!” and only that pet updates its numbers. Ask: why didn’t the other pets change?

Teaching the idea

Real programs store many records with the same shape. A list of dictionaries is a table: each dictionary is one row, and every row has the same keys. Looping over it lets us total, search and rank. A class goes one step further: it is a blueprint that bundles data (attributes) with actions (methods). __init__ runs when an object is built, and self means “this object”, so each pet keeps its own numbers. Keep classes simple today: attributes and methods only.

Analogy: A class is like the design of a Tamagotchi toy in a factory. The design says every toy has hunger and happiness and a feed button. Each toy that comes off the line (an object) has its own numbers, and pressing feed on yours does not feed your friend’s.

Live demo: 1) Type the team list with three dictionaries and print team[0]["name"]. 2) Loop and print each player; then total the runs. 3) Write the top-scorer loop and ask the class to predict the winner. 4) Type class Pet: with __init__ setting self.name and self.hunger; make two pets and print both names. 5) Add def feed(self): and call moti.feed(); print both pets’ hunger to show only Moti changed. 6) Delete self from feed and show the TypeError; put it back.

Questions to ask

Ask	Listen for
What does team[1]["runs"] mean?	The runs of the second player in the list.
How do you find the top scorer?	Keep the best so far and replace it when a player has more runs.
What is the difference between a class and an object?	The class is the blueprint; an object is one thing built from it.

What does self mean inside a method?	The object the method was called on.
When does __init__ run?	Every time a new object is created, like Pet("Moti").

Differentiation

Support
Give the pet starter with __init__ and status complete, so learners write only feed, play, sleep and the loop. Provide a card: method = def name(self): ... called with object.name().

Stretch
Make a list of two pets and let the player choose which one to look after, or add a day counter and make the pet grumpy if hunger reaches 10.

Common misconceptions

Misconception	How to fix it
A method does not need self	Python passes the object in automatically, so every method needs self first.
name = name inside __init__ saves the name	Only self.name = name stores it on the object.
Changing one object changes all of them	Each object has its own attributes.
team["runs"] gives every player's runs	team is a list: pick a player first, team[0]["runs"], or loop.

Assessment

Use the Python Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
In class Pet, which line stores the name on the pet: name = name or self.name = name? Why?

Homework
Teach someone at home to look after your virtual pet. Then add one new method they suggest, like bath() or trick().

Answer key — Kitabash activities

1. Python Lab: team manager (perform · python lab)

Goals checked: Adds up the runs · Finds the top scorer · Prints Zoya as top scorer · Uses loops and return

One solution:

```
team = [
    {"name": "Riya", "runs": 45},
    {"name": "Kabir", "runs": 12},
    {"name": "Zoya", "runs": 67},
]

def total_runs(team):
    total = 0
    for player in team:
        total += player["runs"]
    return total

def top_scorer(team):
    best = team[0]
    for player in team:
        if player["runs"] > best["runs"]:
            best = player
    return best["name"]

print("Total:", total_runs(team))
print("Top scorer:", top_scorer(team))
```

2. Predict the output: the batting object (perform · code output)

3. Build the blueprint (perform · ordering)

Order: class Pet: → def __init__(self, name): → self.name = name → def feed(self): → moti = Pet("Moti") → moti.feed()

4. Project: virtual pet (perform · python lab)

Goals checked: Feeding lowers hunger to 3 · Playing changes happiness and energy · Unknown commands get a message · Uses a class and a game loop

Example program:

```
# Virtual Pet
# Made by: Kabir

class Pet:
    def __init__(self, name):
        self.name = name
        self.hunger = 5
        self.happiness = 5
        self.energy = 5

    def feed(self):
        self.hunger = max(0, self.hunger - 2)
        print(f"{self.name} munches happily.")

    def play(self):
        self.happiness += 2
        self.energy -= 2
        self.hunger += 1
        print(f"{self.name} chases the ball!")

    def sleep(self):
        self.energy += 3
        print(f"{self.name} curls up for a nap. Zzz")

    def mood(self):
        if self.hunger >= 8 or self.energy <= 1:
            return "grumpy"
        elif self.happiness >= 7:
            return "very happy"
        return "okay"

    def status(self):
        print(f"{self.name} | Hunger: {self.hunger} | Happiness: {self.happiness} |
Energy: {self.energy} | Mood: {self.mood()}")

print("=== VIRTUAL PET ===")
pet = Pet(input("Name your pet: ").strip())
playing = True
while playing:
    pet.status()
    cmd = input("feed, play, sleep or quit? ").strip().lower()
    if cmd == "feed":
        pet.feed()
    elif cmd == "play":
        pet.play()
    elif cmd == "sleep":
        pet.sleep()
    elif cmd == "quit":
        playing = False
    else:
```

```
        print("Unknown command. Try feed, play or sleep.")
print(f"Bye! {pet.name} will miss you.")
```

5. Debug it: the forgetful pet (probe · python lab)

Goals checked: Runs without errors · Prints Moti 3

One solution:

```
class Pet:
    def __init__(self, name):
        self.name = name
        self.hunger = 5

    def feed(self):
        self.hunger -= 2

pet = Pet("Moti")
pet.feed()
print(pet.name, pet.hunger)
```

Answer key — worksheet

1. **Key words:** record = One item of data with several parts, like one player's dictionary. · list of dictionaries = A table of records: one dictionary per row. · class = A blueprint that describes data and actions together. · object = One real thing built from a class, like Pet("Moti").
2. **Plan your program:** Goals: Adds up the runs; Finds the top scorer; Prints Zoya as top scorer; Uses loops and return. One solution: team = [{"name": "Riya", "runs": 45}, {"name": "Kabir", "runs": 12}, {"name": "Zoya", "runs": 67},] def total_runs(team): total = 0 for player in team: total += player["runs"] return total def top_scorer(team): best = team[0] for player in team: if player["runs"] > best["runs"]: best = player return best["name"] print("Total:", total_runs(team)) print("Top scorer:", top_scorer(team))
3. **Predict the output: the batting object:** A. Arjun 10
4. **Build the blueprint:** 1 class Pet: · 2 def __init__(self, name): · 3 self.name = name · 4 def feed(self): · 5 moti = Pet("Moti") · 6 moti.feed()
5. **Plan your program:** Goals: Runs without errors; Prints Moti 3. One solution: class Pet: def __init__(self, name): self.name = name self.hunger = 5 def feed(self): self.hunger -= 2 pet = Pet("Moti") pet.feed() print(pet.name, pet.hunger)

Kitabash original curriculum.